

Introduction to Big Data

[illegible]

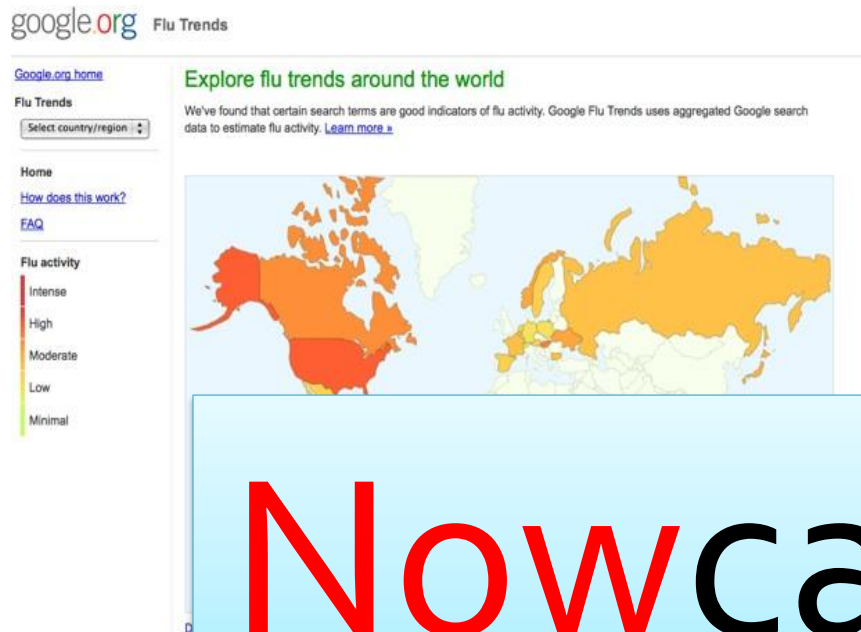
Google Flu trends



- February 2010
 - Google detected flu outbreak two weeks ahead of CDC data (Centers for Disease Control and Prevention – U.S.A)
 - Based on the analysis of Google search queries



Google Flu trends



- February 2010
 - Google detected flu outbreak two weeks ahead of CDC data (Centers for Disease Control and Prevention)

Nowcasting

Nowcasting: using Big Data in order to get quick predictions



Data on the Internet...

■ Internet live stats

- <http://www.internetlivestats.com/>

Big data derive from various sources such as social media, scientific computing systems, log files, IoT... All of them produce large amounts of data which need to be analysed


4,485,508,861
Internet Users in the world


1,752,142,970
Total number of Websites


193,688,718,339
Emails sent *today*


5,222,289,027
Google searches *today*


4,990,992
Blog posts written *today*


572,159,945
Tweets sent *today*


5,348,093,035
Videos viewed *today*
on YouTube


62,832,046
Photos uploaded *today*
on Instagram


107,175,151
Tumblr posts *today*


2,435,900,914
Facebook active users


795,537,418
Google+ active users


357,398,865
Twitter active users


278,573,312
Pinterest active users


287,236,096
Skype calls *today*


110,341
Websites hacked *today*


5,664,059,486 GB
Internet traffic *today*


3,065,544 MWh
Electricity used *today*
for the Internet

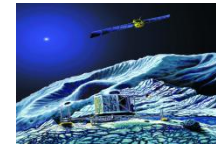

2,507,959 tons
CO₂ emissions *today*

Who generates big data?

- User Generated Content (Web & Mobile)
 - E.g., Facebook, Instagram, Yelp, TripAdvisor, Twitter, YouTube

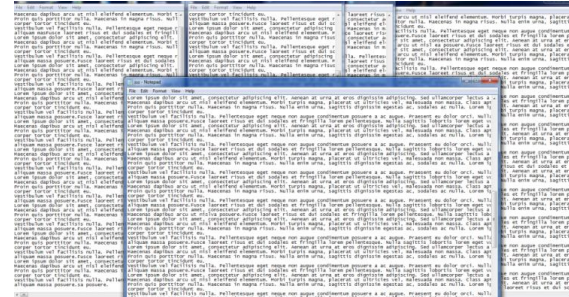


- Health and scientific computing

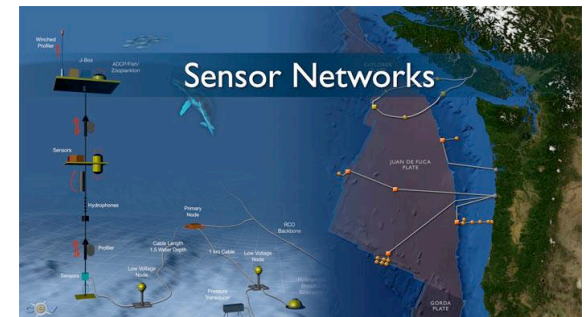
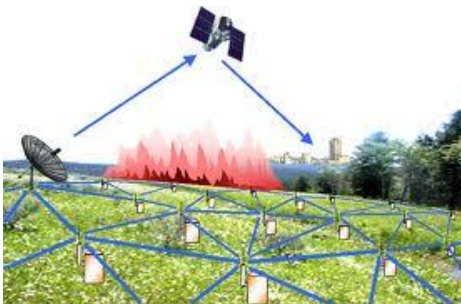


Who generates big data?

- Log files
 - Web server log files, machine system log files



- Internet Of Things (IoT)
 - Sensor networks, RFIDs, smart meters



The Vs of big data

- The 3Vs of big data
 - **Volume**: scale of data
 - **Variety**: different forms of data
 - **Velocity**: analysis of streaming data
- ... but also
 - **Veracity**: uncertainty of data
 - **Value**: exploit information provided by data

↗ speed at which
data is generated

Big Data Frameworks: Motivations and Challenges

Data volumes

- The amount of data increases every day
- Some numbers (~ 2012):
 - Data processed by Google every day: 100+ PB
 - Data processed by Facebook every day: 10+ PB
- To analyze them, systems that scale with respect to the data volume are needed

Data volumes: Google Example

- Analyze **10 billion web pages**
- Average size of a webpage: **20KB**
- Size of the collection: 10 billion x 20KBs = **200TB**
- HDD hard disk read bandwidth: 150MB/sec
- Time needed to **read all web pages (without analyzing them)**: more than **15 days**
- A single node architecture is not adequate

Data volumes: Google Example with SSD

- Analyze **10 billion web pages**
- Average size of a webpage: **20KB**
- Size of the collection: 10 billion x 20KBs = **200TB**
- SSD hard disk read bandwidth: 550MB/sec
- Time needed to **read all web pages (without analyzing them)**: more than **4 days**
- A single node architecture is not adequate

One single server is not enough

Failures

- Failures are part of everyday life, especially in data center
 - A single server stays up for 3 years (~1000 days)
 - 10 servers → 1 failure every 100 days (~3 months)
 - 100 servers → 1 failure every 10 days
 - 1000 servers → 1 failure/day
- Sources of failures
 - Hardware/Software
 - Electrical, Cooling, ...
 - Unavailability of a resource due to overload

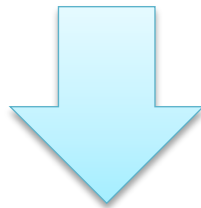
Hardware and software failures are quite common, especially when thousands of servers are used.

Network bandwidth

- Network becomes the bottleneck if big amounts of data need to be exchanged between nodes/servers
 - Network bandwidth (in a data center): 10Gbps
 - Moving 10 TB from one server to another takes more than 2 hours
 - **Data** should be **moved across nodes only** when it is **indispensable**

The solution

- Transfer the **processing power** and **code** to the **data**
- Usually, codes/programs are small (few MBs)
→ **Move code** (programs) and **computation** to data



Data locality

Instead of moving data, we can move the code to data, taking advantage of data locality in order to improve performance.

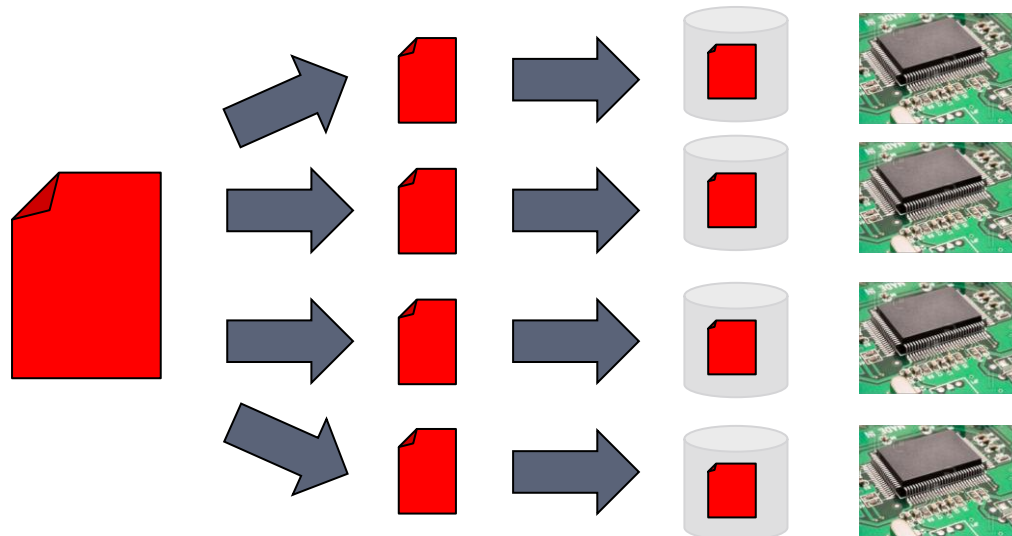
The solution

- **Multiple distributed disks**

- Each one holding a portion of a large dataset

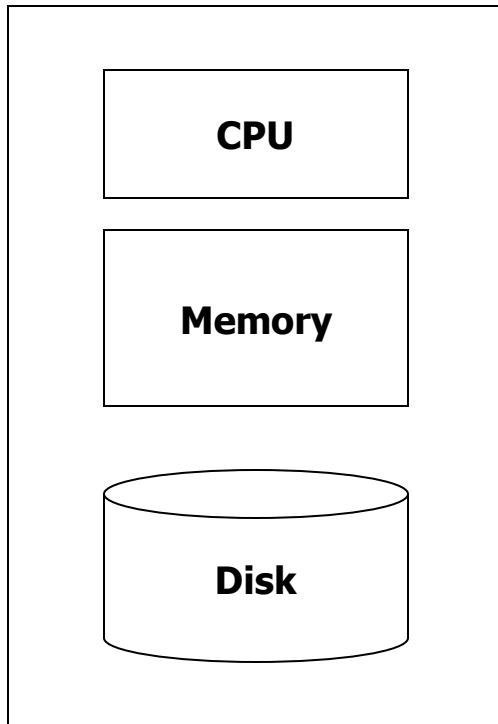
- **Process in parallel different** file **portions** from different disks

We could divide data into multiple disks, granting a parallel analysis.



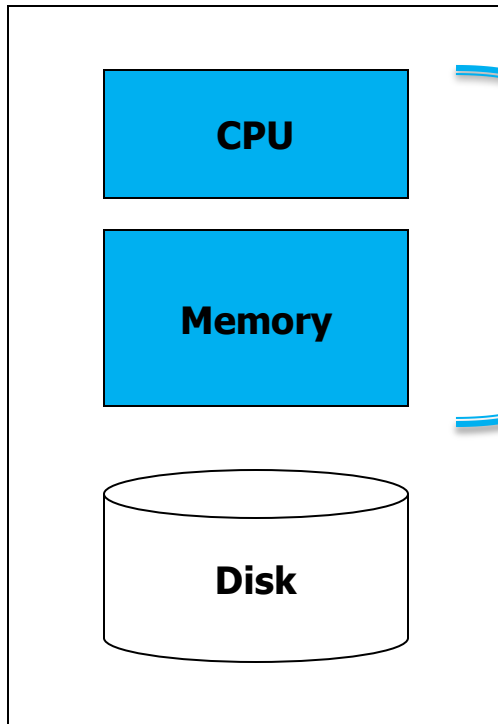
Single-node architecture

Server (Single node)



Single-node architecture

Server (Single node)

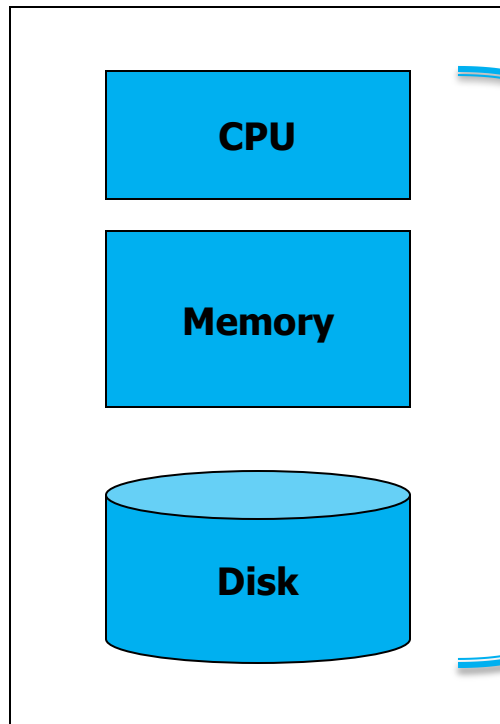


Shallow Machine Learning, Statistics

- Small data
 - Data can be completely loaded in main memory but only if memory is large enough

Single-node architecture

Server (Single node)



"Classical" data mining

- Large data
 - if Data can not be completely loaded in main memory
 - we can Load in main memory one chunk of data at a time
 - Process it and store some statistics
 - Combine statistics to compute the final result

Cluster Architecture

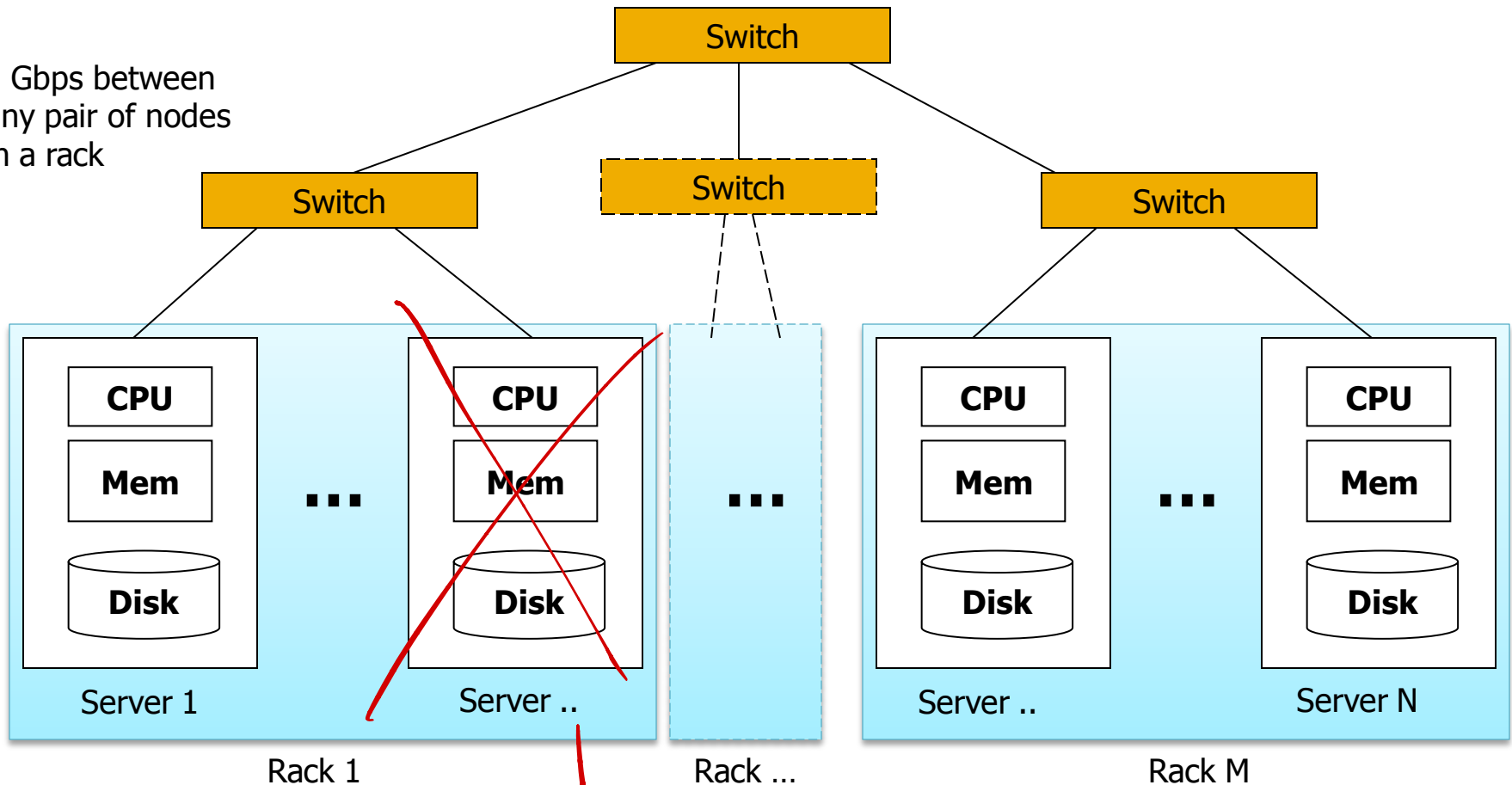
- Cluster of servers (data center)
 - Computation is distributed across servers
 - Data are stored/distributed across servers
- Standard architecture in the Big data context (~ 2012)
 - Cluster of commodity Linux nodes/servers
 - 32 GB of main memory per node
 - Gigabit Ethernet interconnection

We use relatively cheap servers but a great number of servers.

Commodity Cluster Architecture

2-10 Gbps backbone between racks

1 Gbps between
any pair of nodes
in a rack



Each rack contains 16-64 nodes

if it broke just replace it

Data center



Data center



Scalability

- Current systems must scale to ^{satisfy} 
 - The increasing amount of data to analyze
 - The increasing number of users to serve
- Two approaches are usually used to address scalability issues
 - Vertical scalability (scale up)
 - Horizontal scalability (scale out)

Scale up vs. Scale out

- Vertical scalability (scale up)
 - Add more power/resources (main memory, CPUs) to a single node (high-performing server)
 - but ■ Cost of super-computers is not linear with respect to their resources → it could be expensive
- Horizontal scalability (scale out)
 - Add more nodes (commodity servers) to a system
 - The cost scales approximately linearly with respect to the number of added nodes → cheaper but more complex
 - But data center efficiency is a difficult problem to solve

Scale up vs. Scale out

- For **data-intensive workloads**, a **large number of commodity servers** is **preferred** over a small number of high-performing servers
 - At the same cost, we can deploy a system that processes data more efficiently and is more fault-tolerant
- **Horizontal scalability (scale out)** is preferred for **big data** applications
 - **Anyway, it could be complex because of task synchronization or failures**
 - New systems hiding the complexity of the distributed part of the problem to developers are needed

Cluster computing challenges

- Distributed programming is hard
 - Problem decomposition and parallelization
 - Task synchronization
- Task scheduling of distributed applications is critical
 - Assign tasks to nodes by trying to
 - Speed up the execution of the application
 - Exploit (almost) all the available resources
 - Reduce the impact of node failures

Cluster computing challenges

- Distributed data storage
 - How do we store data persistently on disk and keep it available if nodes can fail? ↘ solution
 - Redundancy is the solution, but it increases the complexity of the system
- Network bottleneck
 - Reduce the amount of data send through the network ↘ solution
 - Move computation and code to data

Cluster computing challenges

- Distributed computing is not a new topic
 - **HPC** (High-performance computing) ~1960
 - **Grid computing** ~1990
 - **Distributed databases** ~1990
- Hence, many solutions to the mentioned challenges are already available
- But we are now facing big data driven-problems
 - The former solutions are **not adequate to address big data** volumes

Big Data Challenges: A Summary

- The challenges:
 - Parallelization/Distributed computation
 - Distributed storage of large data sets (Terabytes, Petabytes, ..)
 - Node failure management
 - Network bottleneck
 - Diverse input format (data diversity & heterogeneity)

Existing solutions are not enough to face the growing amount of today's data, making necessary the adoption of new technologies.

Typical Big Data Problem

- Typical Big Data Problem
 - Iterate over a big amount of records/objects
 - Extract something of interest from each record/object
 - Aggregate intermediate results
 - Generate final output/global result